

Case Description

Perusahaan ekspedisi lokal tempat Anda bekerja sedang menghadapi dilema infrastruktur teknologi operasional. Saat ini, penentuan rute kurir pengantaran (*Last-Mile Delivery*) menggunakan algoritma pencarian heuristik dasar. Algoritma ini berjalan sangat cepat dan nyaris tidak memakan biaya server komputasi awan (*Cloud Server*). Namun, kelemahannya adalah rute yang dihasilkan sering kali sub-optimal, menyebabkan kurir menempuh jarak lebih jauh dan menghabiskan lebih banyak bahan bakar (BBM).

Manajemen berencana melakukan *upgrade* arsitektur perangkat lunak dengan mengimplementasikan algoritma eksak tingkat lanjut (seperti variasi *Dynamic Programming* atau *Backtracking*) yang dijamin menghasilkan rute paling efisien secara absolut. Permasalahannya, algoritma eksak untuk masalah penyelesaian rute (TSP) memakan waktu komputasi yang sangat berat secara eksponensial. Hal ini akan menyebabkan lonjakan tagihan *Cloud Server* yang menggunakan sistem *Pay-as-you-go* (dibayar per milidetik eksekusi).

Tugas Anda: Manajemen tidak bisa memutuskan ini hanya dengan teori. Anda ditugaskan untuk membangun *pipeline* simulasi komputasi yang membandingkan kedua arsitektur algoritma tersebut. Anda harus membuktikan pada skenario ekonomi seperti apa investasi pada algoritma mahal ini masuk akal secara finansial bagi perusahaan, dan pada skenario apa perusahaan sebaiknya tetap bertahan menggunakan algoritma lama.

Instructions on What to Do

Sebagai Lead Software Engineer, Anda wajib melakukan langkah-langkah berikut:

1. **Pemodelan Data:** Petakan lokasi Gudang Pusat (Hub) dan minimal 10–12 lokasi pelanggan nyata ke dalam struktur data Graf berbobot (Matriks Ketetanggaan). Bobot antar titik merepresentasikan jarak fisik (km).
2. **Implementasi Multi-Algoritma:** Tulis secara mandiri dua jenis fungsi algoritma pencarian rute:
 - **Algoritma A (Heuristik):** Fokus pada kecepatan eksekusi tinggi (misal: *Greedy*).
 - **Algoritma B (Eksak):** Fokus pada optimasi jarak terbaik (misal: *DFS with Pruning / Backtracking*).

3. **Simulasi Fungsi Biaya (Cost Function):** Hitung variabel dinamis di setiap titik rute, di mana beban paket memengaruhi efisiensi bensin.
4. **Eksekusi Skenario Ekonomi:** Jalankan kedua algoritma tersebut pada dua skenario bisnis:
 - **Skenario Subsidi:** Harga BBM sangat murah (Rp 5.000/liter).
 - **Skenario Krisis:** Harga BBM melambung tinggi (Rp 20.000/liter).
5. **Analisis Keputusan Bisnis:** Hitung *Total Cost of Ownership* (TCO) untuk tiap skenario dan buat kesimpulan bisnis final (algoritma mana yang direkomendasikan pada masing-masing kondisi ekonomi).

Technical Specifications

Kode yang Anda tulis harus mematuhi batasan dan metrik teknis berikut:

A. Metrik Finansial Dasar

Gunakan formula berikut untuk menghitung pengeluaran:

- **Biaya BBM Rute:** $\sum (Jarak_{AB} \times RasioKonsumsiBensin \times HargaBensin)$
 - *Rasio Konsumsi:* Motor yang terisi penuh beban paket berjalan lebih boros. Beban berkurang setiap kali kurir mampir di lokasi pelanggan. Asumsikan rumus rasio sendiri (misal: 1 km membutuhkan 0.05 liter saat penuh, dan 0.02 liter saat kosong).
- **Biaya Komputasi Server:** $WaktuEksekusiAlg \times Rp50$
- **TCO (Total Cost of Ownership):** $BiayaBBM + BiayaKomputasi$

B. Input / Output

- **Input Constraint:** Data lokasi, berat paket, matriks jarak, dan status skenario ekonomi TIDAK BOLEH ditulis *hardcoded* di dalam logika utama. Semua parameter masukan harus dibaca dari file eksternal berektensi *.json* atau *.csv*.
- **Output Constraint:** Output program berjalan di Terminal / *Command Line Interface* (CLI). Wajib menampilkan waktu eksekusi presisi untuk kedua algoritma, urutan rute, dan komparasi nilai TCO dari kedua skenario.

C. Restriksi Algoritmik

- Dilarang keras menggunakan *built-in function* atau *library* pihak ketiga untuk menyelesaikan logika pencarian rute atau graf utama (seperti *networkx* di

Python atau algoritma pencarian bawaan bahasa lainnya). Inti algoritma wajib di-code dari awal (*from scratch*).

Pengumpulan

Seluruh hasil analisis, desain, dan implementasi kode wajib diunggah (*push*) ke repositori GitHub masing-masing praktikan. Repositori tersebut akan menjadi penilaian utama (*Portfolio Base*).

Struktur Repositori Wajib

Repositori harus rapi dan modular. Contoh:

```
|—src/           # File source code utama berisi implementasi algoritma
|—data/         # File masukan (.json/.csv) berisi dataset lokasi & matriks
|—docs/        # Screenshot output terminal atau grafik perbandingan eksekusi
|—README.md    # Dokumentasi dan panduan eksekusi
|—.gitignore   # File konfigurasi git ignore (node_modules, pycache, dsb.)
```

Ketentuan **README.md**

1. **Cara Menjalankan Program:** Instruksi *build* atau *run* (contoh: `python src/main.py --scenario crisis`).
2. **Pemilihan Algoritma:** Penjelasan algoritma eksak dan heuristik apa yang Anda pilih beserta alasan (*trade-off*).
3. **Analisis Kompleksitas (Big-O):** Jabarkan perhitungan teoritis ruang dan waktu dari kedua algoritma tersebut berdasarkan penelusuran struktur *loop* / rekursif.
4. **Summary:** Paragraf kesimpulan keputusan bisnis. Pada harga bensin berapa algoritma eksak mulai menguntungkan untuk dijalankan (Titik *Break-Even*)?

Ketentuan **Commit History**

- Progres pengerjaan juga akan dinilai melalui log *commit*.
- Dilarang melakukan unggahan masif hanya dengan satu *commit* di akhir tugas (misal: `Initial commit` atau `Done`).
- Gunakan pesan *commit* yang bermakna dan deskriptif yang merefleksikan proses *pengerjaan* (contoh: `feat: implement dfs with pruning for exact match`, `fix: correct fuel ratio calculation state`, `docs: update TCO reasoning in readme`).